Robotics and Flexible Automation

Lecture 7
Jacobian, Force and Compliance

Prof. Zengxi (Stephen) Pan

BJTU 2026

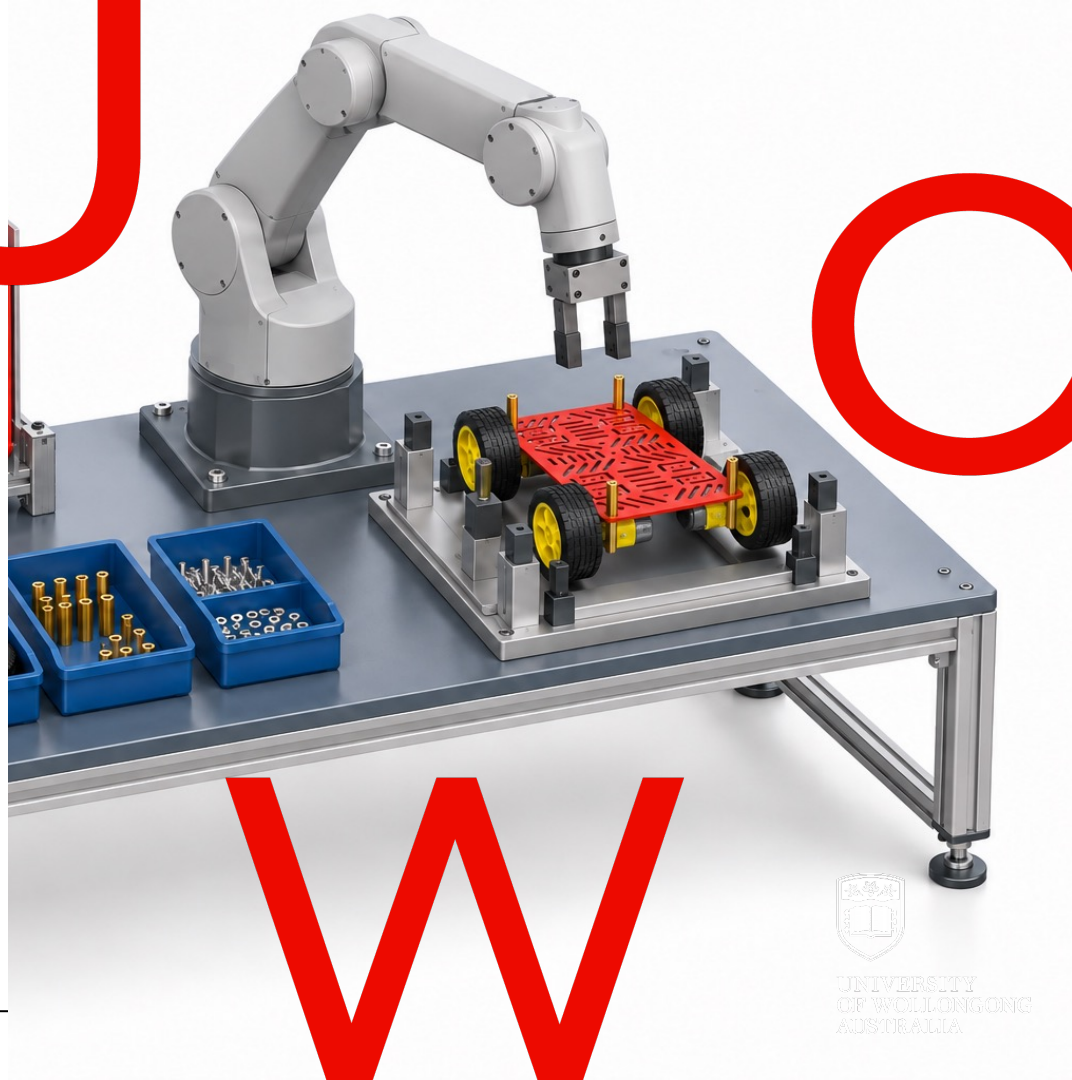


Table of Contents

CORE

1 Jacobian foundations, derivation and examples

2 Velocity and singularity

3 Force, stiffness and compliance

PART 1

Jacobian foundations and worked examples

Relate joint variables to tool motion

1. General and manipulator Jacobians
2. Joint contributions and construction algorithm
3. Worked examples: SCARA and spherical robot
4. Jacobian frames

What this lecture is about

CORE

*Jacobian **J(q)**, connects joint motion, contact force and robot stiffness*

velocity

$$\dot{x} = J(q)\,\dot{q}$$

joint speed → tool speed

force

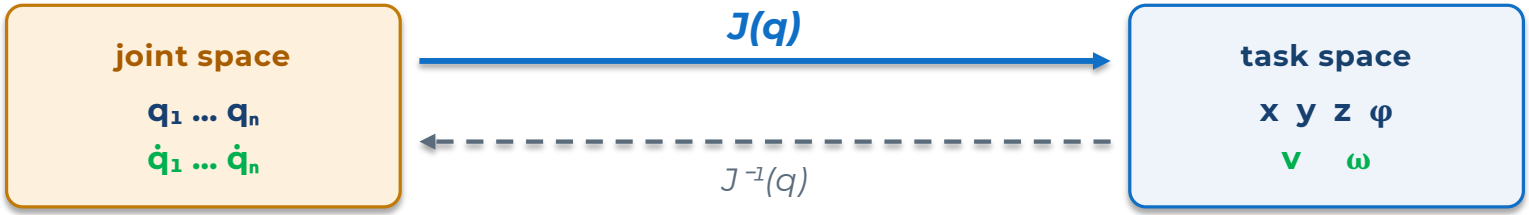
$$\tau = J^T(q)\,F$$

contact wrench → joint torque

stiffness

$$F = K_x\,\Delta x$$

contact force → tool deflection



The Jacobian: the general definition

CORE

Jacobian is the first partial derivatives of a vector function

given m functions of n variables

$$y_1 = f_1(x_1, x_2, \dots, x_n)$$
$$\vdots$$
$$y_m = f_m(x_1, x_2, \dots, x_n)$$

$$\delta y = J\,\delta x\ ,$$

$$J(q) = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}$$

Rows: one output

Row i shows how every input affects output y_i.

Columns: one input

Column j shows how input q_j affects all outputs.

J has m rows and n columns

$$\Delta y \approx J(q)\Delta q$$

$$\dot{y} = J(q)\dot{q}$$

The Jacobian describes local sensitivity at the current configuration.

Analytical and Geometric Jacobians

p = TCP position η = [roll, pitch, yaw]^T q = joint coordinates

Analytical Jacobian

$$\begin{bmatrix} \dot{p} \\ \dot{\eta} \end{bmatrix} = \begin{bmatrix} \frac{\partial p}{\partial q} \\ \frac{\partial \eta}{\partial q} \end{bmatrix} \dot{q} = J_a \dot{q}$$

Geometric Jacobian

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = \begin{bmatrix} J_v \\ J_\omega \end{bmatrix} \dot{q} = J_g \dot{q}$$

For a manipulator, x = f(q). Two Jacobian forms are common:

Manipulator (geometric) Jacobian: $V = J(q)\dot{q}$

- Maps joint rates to TCP linear and angular velocity.
- Used for velocity, singularity, force and stiffness.

Analytical Jacobian: $\dot{x} = J_a(q)\dot{q}$

- Maps joint rates to rates of the selected pose coordinates (RPY angles).
- Arm dynamics

RPY angle rates generally differ from angular velocity. We use the geometric Jacobian.

The Manipulator/Geometric Jacobian

CORE

Small joint motions give instantaneous TCP linear and angular velocity

$$\begin{bmatrix} v_x \\ v_y \\ v_z \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} = \begin{bmatrix} J_v \\ J_\omega \end{bmatrix}(q)\dot{q} = J_g(q)\dot{q}$$

Rows 1–3: TCP linear velocity

Rows 4–6: angular velocity

Column j: joint j contribution

Multiply column j by joint speed $\dot{q}_j$.
Add the contributions of all joints.

6 velocity components × n joint columns

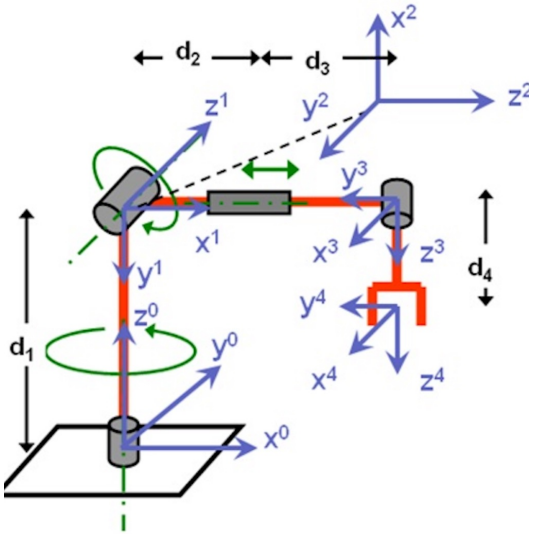
$$\begin{bmatrix} v \\ \omega \end{bmatrix} = J_{g,1}\dot{q}_1 + \cdots + J_{g,n}\dot{q}_n$$

Local motion mapping at the current configuration. All components use the same reference frame.

Robot kinematics:

forward kinematics gives the pose; the Jacobian gives the instantaneous motion

$$ {}^{0}T_{n}(q) = {}^{0}T_{1}(q_1)\ {}^{1}T_{2}(q_2)\ \cdots\ {}^{n-1}T_{n}(q_n) $$



Spherical robot example: complete forward-kinematic result

$$ {}^{0}T_{4} = \begin{bmatrix} S_1C_4 + C_1S_2S_4 & -S_1S_4 + C_1S_2C_4 & -C_1C_2 & -d_4C_1C_2 - q_3C_1S_2 \\ -C_1C_4 + S_1S_2S_4 & C_1S_4 + S_1S_2C_4 & -S_1C_2 & -d_4S_1C_2 - q_3S_1S_2 \\ C_2S_4 & C_2C_4 & S_2 & d_4S_2 - q_3C_2 + d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

Why use joint axes?

- **p(q) is explicit, so A = ∂p/∂q.**
- **R(q) gives orientation, not ω.**
- **We obtain the angular-velocity contribution of each joint from its axis direction, expressed in the base frame.**

Geometric Jacobian: joint contributions

CORE

A(q) maps joint rates to linear velocity. B(q) maps them to angular velocity.

$$J_g(q) = \begin{bmatrix} A(q) \\ B(q) \end{bmatrix} = \begin{bmatrix} J_v \\ J_\omega \end{bmatrix}$$

Translation: A(q) = $J_v(q)$

$$v = A(q)\dot{q}, \qquad A = \frac{\partial p}{\partial q}$$

Rotation: B(q) = $J\omega(q)$

$$\omega = B(q)\dot{q}$$

Joint type	Column A_j: linear contribution	Column B_j: angular contribution
Revolute	$z_{j-1} \times (p - o_{j-1})$	z_{j-1}
Prismatic	z_{j-1}	0

Standard DH: z_{j-1} is joint j's unit axis, o_{j-1} its frame origin, p the TCP. All in the base frame.

Geometric Jacobian: computation

CORE

Accumulate transforms, read the axes, then assemble the velocity mapping

1 Accumulate the DH transforms

$$ {}^{0}T_{0} = I, \qquad {}^{0}T_{j} = {}^{0}T_{j-1} \cdot {}^{j-1}T_{j} $$

2 Read each joint axis and origin in the base frame

$$ {}^{0}T_{j-1} = \begin{bmatrix} \cdots & \cdots & z_{j-1} & o_{j-1} \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad p = {}^{0}T_{n}(1{:}3,4) $$

3 Build the columns and stack the two blocks

$$ A = [a_1 \quad \cdots \quad a_n], \qquad B = [b_1 \quad \cdots \quad b_n], \qquad J_g = \begin{bmatrix} A \\ B \end{bmatrix} $$

Revolute: $b_j = z_{j-1}$. Prismatic: $b_j = 0$. Use the joint-type rules for a_j on the previous slide.

Example: Spherical Robot Link Transforms

EXAMPLE

one 4 × 4 per joint, straight from the DH row

$$ {}^{0}T_1 = \begin{bmatrix} C_1 & 0 & -S_1 & 0 \\ S_1 & 0 & C_1 & 0 \\ 0 & -1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^{1}T_2 = \begin{bmatrix} C_2 & 0 & -S_2 & 0 \\ S_2 & 0 & C_2 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^{2}T_3 = \begin{bmatrix} 0 & 0 & -1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & q_3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^{3}T_4 = \begin{bmatrix} C_4 & -S_4 & 0 & 0 \\ S_4 & C_4 & 0 & 0 \\ 0 & 0 & 1 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

each one is the DH link matrix evaluated on its own row of the table

$$ {}^{0}T_2 = \begin{bmatrix} C_1C_2 & S_1 & -C_1S_2 & 0 \\ S_1C_2 & -C_1 & -S_1S_2 & 0 \\ -S_2 & 0 & -C_2 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^{0}T_3 = \begin{bmatrix} S_1 & C_1S_2 & -C_1C_2 & -q_3C_1S_2 \\ -C_1 & S_1S_2 & -S_1C_2 & -q_3S_1S_2 \\ 0 & C_2 & S_2 & d_1 - q_3C_2 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{0}T_4 = \begin{bmatrix} S_1C_4 + C_1S_2S_4 & -S_1S_4 + C_1S_2C_4 & -C_1C_2 & -d_4C_1C_2 - q_3C_1S_2 \\ -C_1C_4 + S_1S_2S_4 & C_1S_4 + S_1S_2C_4 & -S_1C_2 & -d_4S_1C_2 - q_3S_1S_2 \\ C_2S_4 & C_2C_4 & S_2 & d_1 + d_4S_2 - q_3C_2 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

Joint axes in the base frame

$z_0 = [0, 0, 1]^T$
z_1: column 3 of ${}^{0}T_1$
z_2: column 3 of ${}^{0}T_2$
z_3: column 3 of ${}^{0}T_3$

${}^{0}T_4 = {}^{0}T_3 \cdot {}^{3}T_4$ adds the tool offset d_4 along z3 — its position column is the p(q) used on the next slide

Example: Spherical Robot A and B

Read the position and joint axes from the transforms on the previous slide

Linear block: the result of differentiating p(q)

$$A(q) = \frac{\partial p}{\partial q} = \begin{bmatrix} d_4 S_1 C_2 + q_3 S_1 S_2 & d_4 C_1 S_2 - q_3 C_1 C_2 & -C_1 S_2 & 0 \\ -d_4 C_1 C_2 - q_3 C_1 S_2 & d_4 S_1 S_2 - q_3 S_1 C_2 & -S_1 S_2 & 0 \\ 0 & d_4 C_2 + q_3 S_2 & -C_2 & 0 \end{bmatrix}$$

Angular block: axes from the cumulative transforms

$$B(q) = [z_0 \quad z_1 \quad 0 \quad z_3] = \begin{bmatrix} 0 & -S_1 & 0 & -C_1 C_2 \\ 0 & C_1 & 0 & -S_1 C_2 \\ 1 & 0 & 0 & S_2 \end{bmatrix}$$

Joint 3: $a_3 = z_2$ and $b_3 = 0$. Joint 4: $a_4 = 0$ and $b_4 = z_3$.
Stack A above B to obtain the complete Jacobian on the next slide.

Example: Complete Spherical Robot Jacobian

EXAMPLE

four columns, one per joint; six rows, three linear and three angular

columns **q_1 q_2 q_3 q_4** rows **x y z (linear) Rx Ry Rz (angular)**

$$J(q) = \begin{bmatrix} d_4 S_1 C_2 + q_3 S_1 S_2 & d_4 C_1 S_2 - q_3 C_1 C_2 & -C_1 S_2 & 0 \\ -d_4 C_1 C_2 - q_3 C_1 S_2 & d_4 S_1 S_2 - q_3 S_1 C_2 & -S_1 S_2 & 0 \\ 0 & d_4 C_2 + q_3 S_2 & -C_2 & 0 \\ 0 & -S_1 & 0 & -C_1 C_2 \\ 0 & C_1 & 0 & -S_1 C_2 \\ 1 & 0 & 0 & S_2 \end{bmatrix}$$

column 3 — the prismatic joint: translation only, and it is just z2

column 4 — the tool roll: rotation only, since the tool turns about its own axis

$C_2 = 0$ and $q_3 = 0$ make columns 1 and 4 linearly dependent — the singular pose

Full rank loss requires $C_2 = 0$ AND $q_3 = 0$. Aligned axes alone do not prove full Jacobian rank loss.

Example: SCARA Jacobian

EXAMPLE

two revolute joints, one prismatic joint, one tool roll



q_1, q_2: revolute

q_3: prismatic
q4: revolute

$$ {}^{\text{base}}T_{\text{tool}}(q) = \begin{bmatrix} C_{1-2-4} & S_{1-2-4} & 0 & a_1C_1 + a_2C_{1-2} \\ S_{1-2-4} & -C_{1-2-4} & 0 & a_1S_1 + a_2S_{1-2} \\ 0 & 0 & -1 & d_1 - q_3 - d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

forward kinematics — note C_{1-2} means $\cos(q_1 - q_2)$

columns **q_1 q_2 q_3 q_4** rows **x y z Rx Ry Rz**

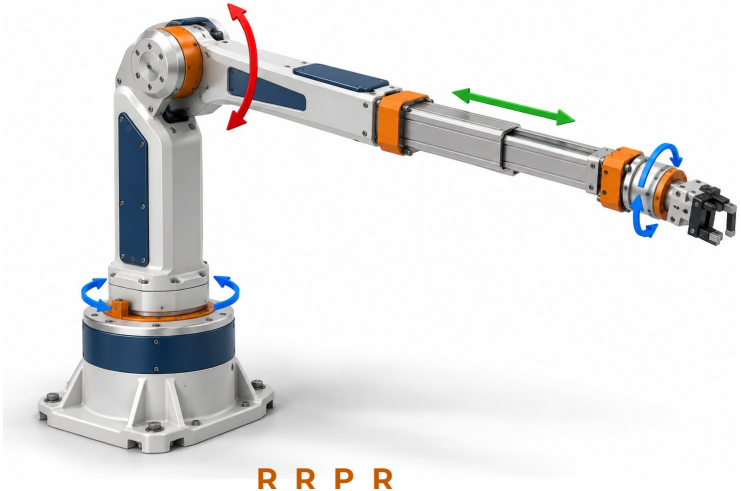
$$ J(q) = \begin{bmatrix} -a_1S_1 - a_2S_{1-2} & a_2S_{1-2} & 0 & 0 \\ a_1C_1 + a_2C_{1-2} & -a_2C_{1-2} & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & -1 & 0 & -1 \end{bmatrix} $$

column 3 is the prismatic joint: a pure translation, no rotation

Example: Spherical Robot

EXAMPLE

waist, shoulder, a prismatic reach and a tool roll — four joints



R R P R

Axis	θ	d	a	α	Home
1	*q1*	*d1*	*0*	*−90°*	*0°*
2	*q2*	*0*	*0*	*−90°*	*−90°*
3	*90°*	*q3*	*0*	*−90°*	*d_2+d3*
4	*q4*	*d4*	*0*	*0°*	*0°*

row 3 carries the prismatic variable q_3 in the d column

$$q = [q_1 \quad q_2 \quad q_3 \quad q_4]^T$$

Joints 1, 2 and 4 are revolute; joint 3 is prismatic, so ζ = (1, 1, 0, 1).

The Jacobian in different frames

CORE

the same physical motion, written in base or in tool coordinates

$$J_n = \begin{bmatrix} ({}^0R_n)^T & 0 \\ 0 & ({}^0R_n)^T \end{bmatrix} J_0$$

R^0_n maps tool-frame coordinates into the base frame; the block diagonal rotates linear and angular velocities separately

J_0 — base frame

natural for workcell geometry and for programming a path

what robot.jacob0(q) returns

J_n — tool frame

natural for contact tasks — "push along the tool z"

what robot.jacobe(q) returns

$$\dot{x}_n = J_n(q)\,\dot{q}$$

$$\tau = J_n^T(q)\,F_n$$

PART 2

Velocity and singularity

Use the Jacobian for joint speeds and recognise rank loss

1. Forward and inverse velocity
2. Boundary and wrist singularities
3. Velocity amplification near singularity

Example: Forward Velocity

EXAMPLE

SCARA: $a_1 = 0.40$ m, $a_2 = 0.30$ m; $q_1 = 30°$, $q_2 = 60°$; $\dot{q}_3 = \dot{q}_4 = 0$

$$\dot{x} = J_0(q)\,\dot{q}$$

quantity	value
J_v row x	[−0.050 −0.150]
J_v row y	[0.606 −0.260]
vx	0.025 m/s
vy	0.736 m/s
ωz	1.500 rad/s
tool speed	0.737 m/s

here $S_{1-2} = \sin(q_1 - q_2) = -0.5$ and $C_{1-2} = 0.866$

```
% bjtu_scara.m supplied with the examples
robot = bjtu_scara();   % 4DOF SCARA
q = [pi/6 pi/3 0 0];
qdot = [1; -0.5; 0; 0];
J0 = robot.jacob0(q);   % 6 x 4
xdot = J0*qdot;
v = xdot(1:3)      % m/s
w = xdot(4:6)      % rad/s
```

Joints 1 and 2 turn; joints 3 and 4 stay fixed. Their velocity columns partly cancel in x. The result is v ≈ (0.025, 0.736, 0) m/s and ωz = 1.5 rad/s.

Inverse velocity: from tool speed to joint speed

CORE

invert the Jacobian when it is square and non-singular; use the pseudoinverse otherwise

square and invertible

$$\dot{q} = J^{-1}(q)\,\dot{x}$$

more joints than task directions

$$\dot{q} = J^{\dagger}(q)\,\dot{x} = J^{T}(JJ^{T})^{-1}\,\dot{x}$$

```
% robot and q from slide 18
xdot = [0; .30; 0; 0; 0; 0];
J0 = robot.jacob0(q);      % 6 x 4
qdot = pinv(J0)*xdot;
% qdot = [.433; -.144; 0; .577]
assert(norm(J0*qdot-xdot) < 1e-9);
```

A SCARA controls v_x, v_y, v_z and ω_z: its task Jacobian has four rows.

Pure +y translation requires joint 4 to cancel the yaw caused by joints 1 and 2.

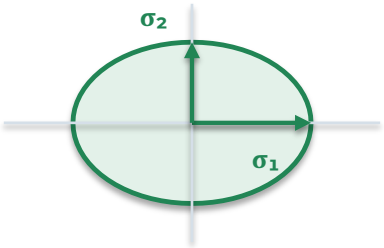
Rates for joints 1, 2 and 4 are rad/s; joint 3 is m/s.

A true inverse needs a full-rank square J. pinv still exists at rank loss, but task motion may be lost.

Singularity: loss of Jacobian rank

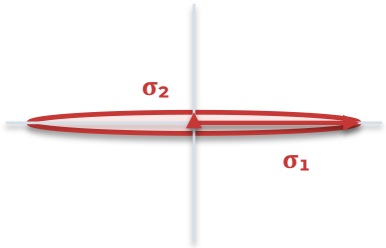
CORE

at a singular pose the robot cannot produce tool motion in at least one direction



well conditioned

$\sigma_1 \approx \sigma_2$ — the tip can move freely in every direction



near singular

$\sigma_2 \to 0$ — one direction of tool motion is unreachable

$$detJ(q) = 0$$

square Jacobian

$$\text{rank}\, J\,(q) < r_0$$

normal task rank r_0

$$\sigma_{min} \to 0$$

the practical test

Example: SCARA Boundary Singularities

EXAMPLE

Task velocity $[v_x, v_y, v_z, \omega_z]^T = J_{task}\,[\dot{a}_1, \dot{a}_2, \dot{a}_3, \dot{a}_4]^T$. $a_1 = 0.40$ m, $a_2 = 0.30$ m, $a_3 = 0°$.



Regular: $q_2 = 90°$

$$J_{task} = \begin{bmatrix} 0.30 & -0.30 & 0 & 0 \\ 0.40 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 1 & -1 & 0 & -1 \end{bmatrix}$$

rank = 4

Extended: $q_2 = 0°$

$$J_{task} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0.70 & -0.30 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 1 & -1 & 0 & -1 \end{bmatrix}$$

rank = 3

Folded: $q_2 = 180°$

$$J_{task} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0.10 & 0.30 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 1 & -1 & 0 & -1 \end{bmatrix}$$

rank = 3

At both boundaries, the first row becomes zero: radial velocity v_x cannot be produced.

Extended pose: $\dot{q} = [0.3, 0.7, 0, -0.4]^T$ gives J_task $\dot{q} = 0$. Columns 1, 2 and 4 become linearly dependent.

The six-axis wrist singularity

CORE

At $q_5 = 0$, wrist axes 4 and 6 align and one independent angular direction is lost



Standard DH: joint axes 4/6 are z_3/z_5. $J\omega$ is wrist-only.

$$q_5 = 0 \Rightarrow z_3 \parallel z_5$$

$$rank\, J_\omega = 2 < 3$$

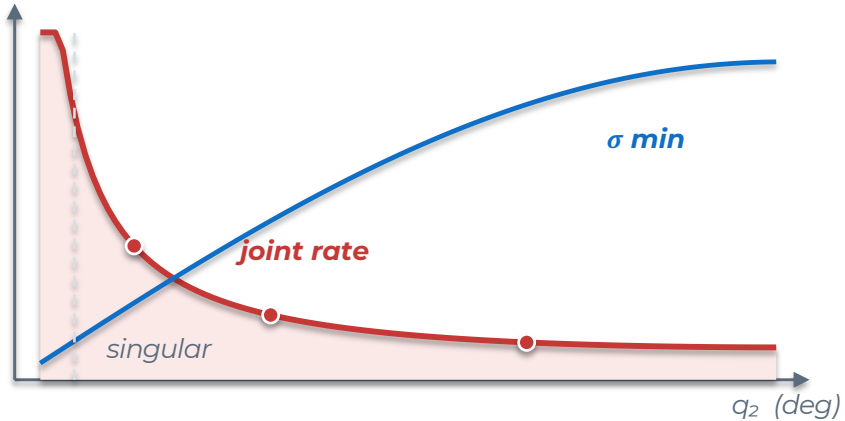
Common-axis rotation remains possible; one independent wrist rotation direction is lost.

Velocity amplification near a singularity

CORE

a modest tool speed can demand a very large joint speed

joint rate needed to hold a constant tool speed of 0.2 m/s



the same tool speed costs more and more joint speed as the pose approaches the singularity

$$\dot{q}_{max} \approx \frac{v_{tip}}{\sigma_{min}}$$

$$\kappa(J) = \frac{\sigma_{max}}{\sigma_{min}} \qquad w(q) = \sqrt{det(J\,J^T)}$$

condition number — manipulability

```
% robot and q from slide 18
dof = [1 1 1 0 0 1];
m = robot.maniplty(q,'dof',dof);
if m < 0.02   % illustrative threshold
    warning('near task singularity');
end
```

Use the robot's available task DOFs; the threshold is model-specific.

PART 3

Force stiffness and compliance

Map forces through the robot and control contact tasks

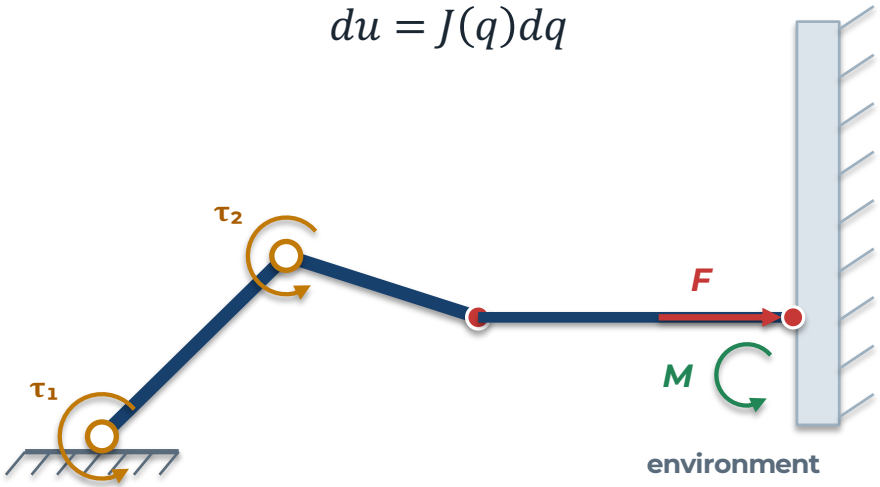
1 Cartesian forces and joint torques
2 Joint and Cartesian stiffness
3 Compliance coupling and contact control

Cartesian forces and joint torques

CORE

Definition of Jacobian

$$du = J(q)dq$$



No dynamics here: this is the static balance between an external wrench and the joint efforts holding against it.

principle of virtual work

$$F^T\, du = \tau^T\, dq$$

substitute du = J dq; it holds for every dq, so

$$F^T J(q)\, dq = \tau^T\, dq$$

$$\tau = J^T(q)\, F$$

the Cartesian wrench: three forces and three moments

$$F = [F_x \quad F_y \quad F_z \quad M_x \quad M_y \quad M_z]^T$$

joint effort vector: revolute joint torque, prismatic joint force

$$\tau = [\tau_1 \quad \tau_2 \quad \cdots \quad \tau_n]^T$$

Example: SCARA Joint Forces and Torques

EXAMPLE

a tool-tip wrench resolved onto the four joints

$$\boldsymbol{\tau} = J^T(q)\mathbf{F} = \begin{bmatrix} -(a_1 S_1 + a_2 S_{1-2}) & a_1 C_1 + a_2 C_{1-2} & 0 & 0 & 0 & 1 \\ a_2 S_{1-2} & -a_2 C_{1-2} & 0 & 0 & 0 & -1 \\ 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 \end{bmatrix} \mathbf{F}$$

This is $J^T(q)$ from the SCARA example on slide 14.

τ_3 is a force, not a torque — joint 3 is prismatic, so it carries the vertical load directly

$$\tau_2 = a_2 S_{1-2} F_x - a_2 C_{1-2} F_y - M_z \qquad \tau_3 = -F_z \qquad \tau_4 = -M_z$$

a moment Mz about the tool axis is shared between joints 1, 2 and 4, because all three turn about a vertical axis

Sizing a joint means checking these numbers at the worst pose in the working envelope, not at the home position.

Joint-space stiffness

CORE

model the whole structure as a spring at each joint and rigid links between them



Joint springs Kq

stiffness — torque needed per unit of deflection

$$\tau = K_q \, \Delta q$$

compliance — deflection produced per unit of torque

$$\Delta q = {K_q}^{-1} \tau = C_q \, \tau$$

$$K_q = diag(k_{q1}, k_{q2}, k_{q3}, \ldots, k_{qn})$$

diagonal: in joint space the joints are decoupled, so each joint deflects only under its own torque

Joint stiffness is a property of the gearbox, bearings and drive train. It does not change with the pose — which is exactly what makes the next slide interesting.

Cartesian stiffness and compliance

CORE

Small deflections about an unloaded pose; joint-spring model only

Jacobian

$$\Delta u = J\,\Delta q$$

virtual work

$$\tau = J^T F$$

joint stiffness

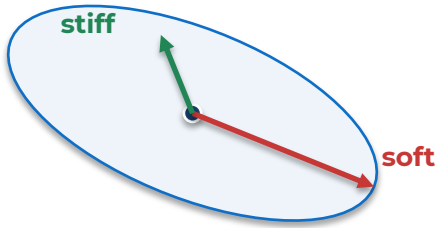
$$\tau = K_q\,\Delta q$$

Cartesian stiffness

$$F = (J^{-T} K_q J^{-1})\Delta u = K_x\,\Delta u$$

Cartesian compliance

$$\Delta u = (J\,{K_q}^{-1} J^T)F = C_x\,F$$



same force
different Δu in each direction

K_x: square, full-rank task J
C_x: invertible Kq; any task J

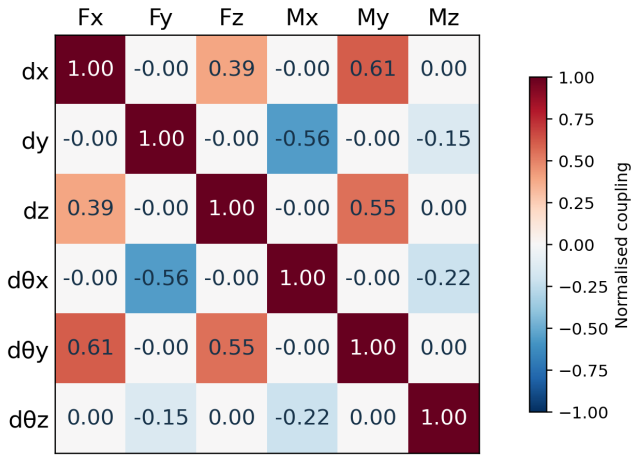
$C_x = J\,Kq^{-1}\,J^T$
the Cartesian compliance matrix

Example: Compliance Matrix Coupling

EXAMPLE

a force in one direction produces deflection in several

$$\Delta u = J(q)\, {K_q}^{-1}\, J^T(q)\, F^{tool}$$



Normalised C_x: symmetric about the diagonal.

IRB2600ID · flange/tool frame
q (deg): [0, −30, 45, 0, 60, 0]
Assumed Kq diagonal (N·m/rad): [100000, 100000, 80000, 30000, 20000, 10000]

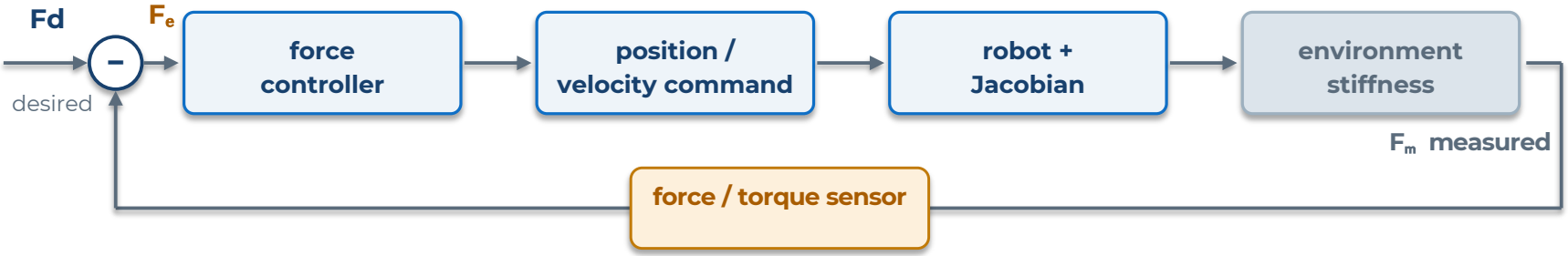
10 N in tool X: 0.067 mm X deflection and 0.134 mrad rotation.

C_x is symmetric; coupling depends on pose.

Force and compliance control

CORE

close a loop on the measured contact force instead of on position alone



$$F_e = F_d - F_m$$

all three use $\tau = J^T F$ from slide 25 — the task is set at the tool, the controller acts on the joints

direct force control

$$\Delta x_c = K_f F_e$$

command a position offset from the force error

impedance control

$$M_d \ddot{e} + B_d \dot{e} + K_d e = -F_{ext}$$

make the tool behave like a chosen mass, damper and spring

admittance control

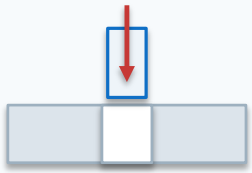
$$\dot{x}_c = C_a F_e$$

measure the force, command the velocity it implies

Contact tasks, and what to take away

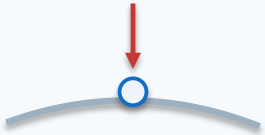
CORE

position-controlled directions, force-controlled directions, and one matrix behind both



peg into hole

let the part find the chamfer instead of fighting it



surface finishing

hold contact force while the surface height varies



assembly / snap fit

detect the insertion force, stop on the threshold

position control along the free directions

force control along the constrained ones

a contact threshold to detect touch, a force limit to stop on a jam

$$\dot{x} = J(q)\,\dot{q}$$ joint speed → tool speed

$$\tau = J^T(q)\,F$$ tool wrench → joint torque

$$\Delta u = J\,K_q{}^{-1} J^T\,F$$ tool wrench → tool deflection

Same J(q) in all three — build it once.


UNIVERSITY
OF WOLLONGONG
AUSTRALIA
Stand
for purpose
THANK YOU!